Java Backend Architecture

Interview Series

Chapter 12.5
Timezone Handling in APIs & Databases

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

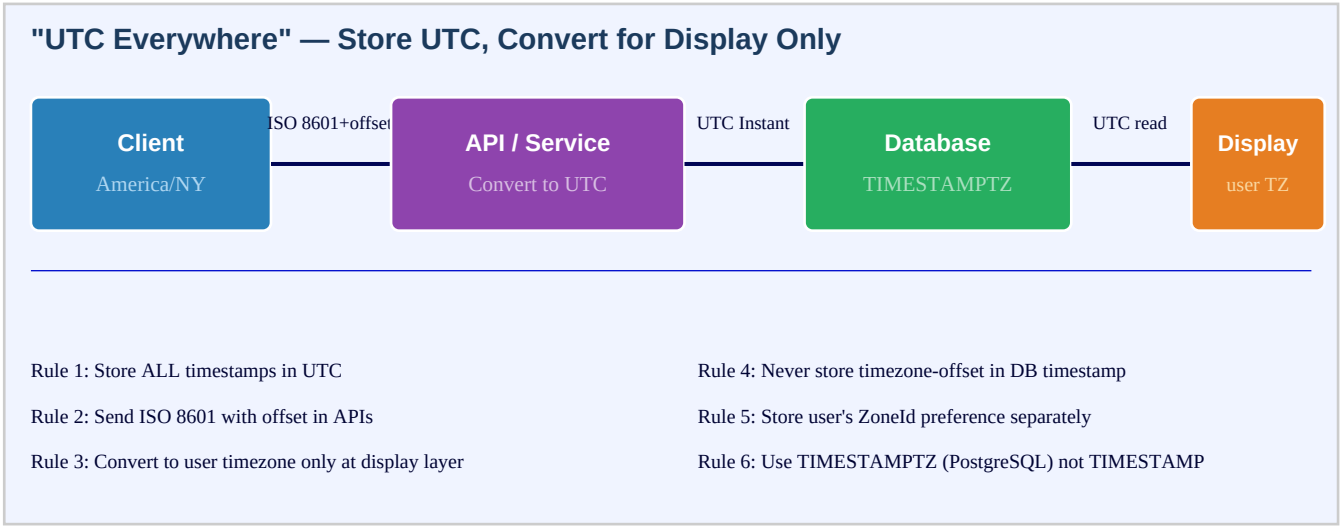
Chapter 12.5 – Timezone Handling in APIs & Databases

Introduction

Timezone bugs are among the most insidious in backend engineering — they are silent, intermittent (they surface only at DST transitions or across geographic boundaries), and have real consequences: appointments at the wrong time, transactions recorded an hour late, financial calculations off by one day. The golden rule is simple: store everything in UTC, convert to the user's local timezone only at the display layer, and never let a timezone assumption leak into business logic or database storage. Java's ZoneId.of("America/New_York") uses the IANA timezone database, which includes the complete history of every timezone rule change and DST transition.

Database timezone handling is equally critical. PostgreSQL's TIMESTAMPTZ normalises all stored values to UTC and converts on retrieval based on the session timezone — set it to UTC in the connection URL to avoid surprises. MySQL's TIMESTAMP also stores in UTC but has the year 2038 problem; DATETIME stores no timezone at all, making it unsafe for global applications. JDBC 4.2+ supports java.time types directly, eliminating the legacy Calendar-parameter dance. Injecting a Clock bean enables deterministic testing of timezone-sensitive logic without mocking static methods.

"UTC Everywhere" Architecture



DST Pitfalls: Spring Forward & Fall Back

DST Pitfalls: Spring Forward & Fall Back (US Eastern, 2024)

Spring Forward: Mar 10, 2024 2:00am → 3:00am

01:59:59 EST (-05:00) exists

02:30:00 AM does NOT exist → gap!

Fall Back: Nov 3, 2024 2:00am → 1:00am

01:30:00 happens TWICE: EDT (-04) and EST (-05)

Ambiguous without offset! 60 min overlap

Duration.between() vs ZonedDateTime arithmetic across DST:

Instant: 2:00 EST + 24h = 2:00 EST next day (correct for "elapsed time")

ZonedDateTime: "tomorrow at 2pm" in NY = same wall clock, different offset (correct for scheduling)

Fix: OffsetDateTime in APIs avoids ambiguity; store ZoneId for recurring schedules.

Database Timezone Handling

Database Timezone Handling: PostgreSQL vs MySQL

PostgreSQL

- TIMESTAMPTZ: stores UTC, shows per session TZ
- TIMESTAMP: no TZ (naive) — avoid for global apps
- SET timezone = 'UTC'; in connection/config
- now() returns current TIMESTAMPTZ

MySQL

- TIMESTAMP: UTC stored, session-TZ displayed (2038!)
- DATETIME: no TZ at all; literal as entered
- SET time_zone = '+00:00'; per session
- TIMESTAMP range: 1970-2038 (year 2038 problem)

Best Practice: JDBC + Spring/JPA

```
spring.jpa.properties.hibernate.jdbc.time_zone=UTC
```

Map: Instant / OffsetDateTime (not java.util.Date / Calendar)

JDBC 4.2+: pstmt.setObject(n, offsetDateTime) — use java.time types directly

Key Definitions

UTC	Coordinated Universal Time — the primary time standard; no DST; offset = +00:00; use for all storage
ZoneId	IANA timezone identifier: "America/New_York", "Asia/Tokyo"; encodes all DST history and future changes
ZoneOffset	Fixed UTC offset: "+05:30", "-08:00"; no DST rules; used in OffsetDateTime
ZonedDateTime	Date-time with ZoneId; DST-aware; correct for user-facing scheduling with future DST changes
OffsetDateTime	Date-time with fixed ZoneOffset; no DST rules; preferred in API payloads for clarity
DST	Daylight Saving Time — clocks spring forward (gap) or fall back (overlap) seasonally in many regions
Spring forward	Clocks jump from 2:00am to 3:00am; 60 minutes of wall-clock time do not exist; some times invalid

Fall back	Clocks roll back from 2:00am to 1:00am; 60 minutes of wall-clock time occur twice; times are ambiguous
TIMESTAMPTZ	PostgreSQL column type: stores as UTC, displays per session timezone; preferred for all timestamps
DATETIME (MySQL)	MySQL column type that stores date+time with no timezone info; literal value; unsafe for global apps
TIMESTAMP (MySQL)	MySQL type: stored in UTC, displayed in session timezone; limited to 1970-2038 (year 2038 problem)
Clock injection	Pattern: inject java.time.Clock into services; Clock.fixed() in tests for deterministic datetime testing
IANA tz database	Internet Assigned Numbers Authority timezone database; Java uses it for ZoneId rules; update JDK for new rules
Joda-Time	Pre-Java-8 date/time library that inspired java.time; now legacy; migrate to java.time
hibernate.jdbc.time_zone	Hibernate property: set to UTC so JDBC sends/receives all timestamps in UTC, not JVM default timezone

Database Timestamp Types Comparison

Database	Type	Timezone	Java Mapping	Recommendation
PostgreSQL	TIMESTAMPTZ	Stored UTC, displayed per session	Instant / OffsetDateTime	USE THIS
PostgreSQL	TIMESTAMP	No timezone (naive)	LocalDateTime	Avoid for global
MySQL	TIMESTAMP	UTC stored, session TZ display	Instant	Beware 2038
MySQL	DATETIME	No timezone at all	LocalDateTime	Avoid for global
Oracle	TIMESTAMP WITH TZ	Stores offset	OffsetDateTime	OK
H2 / HSQL	TIMESTAMP WITH TZ	UTC	OffsetDateTime	For tests

Code Examples

1. ZoneId, ZonedDateTime and converting to UTC

```
import java.time.*; // Creating zone-aware datetimes ZoneId newYork =
ZoneId.of("America/New_York"); ZoneId tokyo = ZoneId.of("Asia/Tokyo"); ZoneId utc =
ZoneOffset.UTC; // Current time in different zones ZonedDateTime nowNY =
ZonedDateTime.now(newYork); ZonedDateTime nowTokyo = ZonedDateTime.now(tokyo); Instant nowUtc
= Instant.now(); System.out.println(nowNY); // 2024-01-15T10:30:00-05:00[America/New_York]
System.out.println(nowTokyo); // 2024-01-16T00:30:00+09:00[Asia/Tokyo] // Convert user input
(local time + zone) to UTC for storage LocalDateTime userInput = LocalDateTime.of(2024, 1, 15,
10, 30); // user entered "Jan 15 10:30am" ZoneId userZone = ZoneId.of("America/Chicago");
Instant utcForStorage = userInput.atZone(userZone).toInstant(); // 2024-01-15T16:30:00Z //
Convert UTC back to user's timezone for display ZonedDateTime displayTime =
utcForStorage.atZone(userZone); System.out.println(displayTime); //
2024-01-15T10:30:00-06:00[America/Chicago] // Available zone IDs Set<String> zones =
ZoneId.getAvailableZoneIds(); zones.stream().sorted().limit(5).forEach(System.out::println);
// Africa/Abidjan, Africa/Accra, Africa/Addis_Ababa...
```

2. DST-safe arithmetic — Instant vs ZonedDateTime

```
// DST Spring Forward: March 10, 2024 in America/New_York // At 2:00am clocks jump to 3:00am →
"2:30am" does not exist ZoneId ny = ZoneId.of("America/New_York"); // Using Instant (elapsed
time — always 24h = 86400s) Instant beforeDst = ZonedDateTime.of(2024, 3, 10, 1, 0, 0, 0,
ny).toInstant(); Instant after24h = beforeDst.plus(Duration.ofHours(24)); ZonedDateTime
displayAfter = after24h.atZone(ny); System.out.println(displayAfter); //
2024-03-11T02:00:00-04:00 (1am, not 2am — 23 hours wall clock!) // Using ZonedDateTime
(wall-clock time — scheduling semantics) ZonedDateTime wallClock = ZonedDateTime.of(2024, 3,
10, 1, 0, 0, 0, ny); ZonedDateTime next24hWall = wallClock.plusHours(24);
System.out.println(next24hWall); // 2024-03-11T01:00:00-04:00 (same wall clock, 23h elapsed)
// For recurring meetings: "every day at 9am New York" ZonedDateTime meeting =
ZonedDateTime.of(2024, 3, 10, 9, 0, 0, 0, ny); ZonedDateTime nextMeeting = meeting.plusDays(1);
// March 11 9:00am EDT (-04:00) // Even across DST: next meeting is still at 9:00 local, but 1
hour less elapsed time // Gap: "2024-03-10T02:30:00" in NY doesn't exist ZonedDateTime gap =
LocalDateTime.of(2024, 3, 10, 2, 30).atZone(ny); System.out.println(gap); // ZonedDateTime
adjusts to 3:30am EDT automatically
```

3. PostgreSQL TIMESTAMPTZ and JDBC timezone configuration

```
# application.properties
spring.datasource.url=jdbc:postgresql://localhost/mydb?currentSchema=public # Set session
timezone to UTC at connection time: spring.datasource.hikari.connection-init-sql=SET TIME ZONE
'UTC' # Or via Hibernate: spring.jpa.properties.hibernate.jdbc.time_zone=UTC # Flyway
migration: # CREATE TABLE events ( # id BIGSERIAL PRIMARY KEY, # title VARCHAR(255) NOT NULL, #
start_time TIMESTAMPTZ NOT NULL, -- stores UTC, displays in session TZ # event_date DATE NOT
NULL -- no timezone, just a calendar date # ); // JPA Entity @Entity @Table(name = "events")
public class Event { @Id @GeneratedValue private Long id; private String title; @Column(name =
"start_time") private OffsetDateTime startTime; // maps to TIMESTAMPTZ @Column(name =
"event_date") private LocalDate eventDate; // maps to DATE // Store in UTC: public void
setStartTimeInZone(LocalDateTime localTime, ZoneId zone) { this.startTime =
localTime.atZone(zone) .toOffsetDateTime() .withOffsetSameInstant(ZoneOffset.UTC); } //
Display in user zone: public ZonedDateTime getStartTimeInZone(ZoneId zone) { return
startTime.atZoneSameInstant(zone); } }
```

4. ISO 8601 with offset in API responses

```
// REST API: always return ISO 8601 with UTC offset @RestController
@RequestMapping("/api/events") public class EventController { @GetMapping("/{id}") public
EventDto getEvent(@PathVariable Long id, @RequestHeader(value = "X-User-Timezone",
defaultValue = "UTC") String tzHeader) { Event event = eventService.findById(id); ZoneId
userZone = ZoneId.of(tzHeader); // e.g. "America/Chicago" return EventDto.builder()
.id(event.getId()) .title(event.getTitle()) // Return UTC timestamp — client can display in its
own timezone .startTimeUtc(event.getStartTime().toString()) // "2024-01-15T10:30:00Z" // Also
return local time for convenience .startTimeLocal(event.getStartTime()
.atZoneSameInstant(userZone) .toString()) // "2024-01-15T04:30:00-06:00"
.timezone(userZone.getId()) .build(); } @PostMapping public EventDto createEvent(@RequestBody
CreateEventRequest req) { // req.startTime = "2024-01-15T10:30:00-05:00" (client sends with
offset) OffsetDateTime odt = OffsetDateTime.parse(req.getStartTime()); // Normalise to UTC for
storage OffsetDateTime utc = odt.withOffsetSameInstant(ZoneOffset.UTC); // ... } } // Never
return: "January 15, 2024 10:30 AM" (locale-formatted, no timezone) // Never return:
1705315800000 (epoch millis — hard to debug, no timezone) // Always return:
"2024-01-15T10:30:00Z" or "2024-01-15T10:30:00-05:00"
```

5. Storing user timezone preference and Clock injection

```
// User entity: store timezone preference @Entity public class User { @Id private Long id;
private String email; @Column(name = "timezone_id") private String timezoneId = "UTC"; // e.g.
"America/New_York" public ZoneId getZoneId() { try { return ZoneId.of(timezoneId); } catch
(DateTimeException e) { return ZoneOffset.UTC; // fallback if stored timezone is invalid } } }
// Clock injection for testability @Configuration public class ClockConfig { @Bean public Clock
clock() { return Clock.systemUTC(); // production } } @Service public class BookingService {
private final Clock clock; private final BookingRepository repo; public Booking
createBooking(CreateBookingCmd cmd, User user) { Instant now = clock.instant(); // ← testable;
no Instant.now() static call ZoneId userZone = user.getZoneId(); LocalDateTime localStart =
cmd.getStartTime(); Instant startInstant = localStart.atZone(userZone).toInstant(); if
(startInstant.isBefore(now)) { throw new PastBookingException("Cannot book in the past"); }
return repo.save(new Booking(startInstant, user.getId())); } } // Test: fixed clock
@SpringBootTest class BookingServiceTest { @Autowired private BookingService service; @Bean
@Primary // override production Clock public Clock fixedClock() { return
Clock.fixed(Instant.parse("2024-01-15T10:00:00Z"), ZoneOffset.UTC); } }
```

6. Migrating from Joda-Time and java.util.Date to java.time

```
// Migration from legacy java.util.Date java.util.Date legacyDate = new java.util.Date(); // →
Instant Instant instant = legacyDate.toInstant(); // → ZonedDateTime ZonedDateTime zdt =
legacyDate.toInstant().atZone(ZoneId.systemDefault()); // → LocalDate (strip time, use system
zone) LocalDate date = legacyDate.toInstant() .atZone(ZoneId.of("America/New_York"))
.toLocalDate(); // From java.sql.Timestamp (Hibernate legacy) java.sql.Timestamp ts =
java.sql.Timestamp.valueOf("2024-01-15 10:30:00"); Instant fromSql = ts.toInstant(); // ←
toInstant() is timezone-safe // OR: ts.toLocalDateTime() ← uses JVM default zone — risky! //
Joda-Time migration // org.joda.time.DateTime → java.time.ZonedDateTime // new
org.joda.time.DateTime() → ZonedDateTime.now() // jodaDateTime.toInstant().toEpochMilli() →
instant.toEpochMilli() // jodaDateTime.withZone(DateTimeZone.UTC) →
zdt.withZoneSameInstant(ZoneOffset.UTC) // TimeZone.getDefault() — avoid! //
java.util.TimeZone tz = TimeZone.getTimeZone("America/New_York"); // legacy //
java.time.ZoneId zone = tz.toZoneId(); // bridge method // Risk: TimeZone.getDefault() returns
JVM default set at startup // → different on dev laptop (Europe/London) vs server (UTC) →
silent bugs // Fix: always pass explicit ZoneId, never rely on default
```

Interview Questions & Answers

Q1. What is the 'UTC everywhere' principle and why is it important?

Store all timestamps in UTC in the database; convert to the user's local timezone only at the display layer. Importance: (1) Comparison and arithmetic on UTC timestamps is always correct — no DST offsets to account for. (2) A database in UTC is readable by any developer anywhere in the world. (3) When users travel or change their timezone preference, historical timestamps remain accurate — only the display changes. (4) No ambiguity during DST transitions — UTC has no 'fall back'. Violations: storing local timestamps means queries like 'events in the last 24 hours' become incorrect across DST transitions.

Q2. Explain the difference between ZoneId and ZoneOffset.

ZoneOffset is a fixed offset from UTC: +05:30, -08:00. It never changes — no DST. ZoneId is an IANA timezone identifier: 'America/New_York', 'Europe/Paris'. It includes the full history and future of all offset changes (DST rules, historical changes). In summer, America/New_York is -04:00 (EDT); in winter, -05:00 (EST). ZoneId is necessary for correct future-date calculations. ZoneOffset is sufficient for API responses (explicit, no hidden rules). Store ZoneId in the database (as a string column) for recurring schedules; use

ZoneOffset in OffsetDateTime for archived timestamps.

Q3. What happens during a DST 'spring forward' and how should your code handle it?

Spring forward: clocks jump from 2:00am to 3:00am (US Eastern, second Sunday March). The 60 minutes from 2:00am to 3:00am don't exist on that day. If code tries to create ZonedDateTime.of(2024, 3, 10, 2, 30, 0, 0, nyZone), Java automatically adjusts to 3:30am. For scheduling: if a user sets a daily alarm for 2:30am, skip it on the gap day or adjust to 3:30am (document which). For elapsed time calculations: use Instant + Duration — this counts actual seconds, not wall-clock hours. Test: use Clock.fixed(Instant) set to just before the DST transition.

Q4. What happens during DST 'fall back' and how do you resolve ambiguous times?

Fall back: clocks roll from 2:00am to 1:00am (US Eastern, first Sunday November). The period from 1:00am to 2:00am happens twice — once in EDT (-04:00) and once in EST (-05:00). A time of '1:30am' on that day is ambiguous without an offset. ZonedDateTime resolves ambiguity with the withEarlierOffsetAtOverlap() or withLaterOffsetAtOverlap() methods. For scheduling: recurring jobs running 'at 1:30am' may fire twice if not guarded. For API input: require the offset '2024-11-03T01:30:00-04:00' or '2024-11-03T01:30:00-05:00' — never accept a timezone-free local time for this.

Q5. What is the difference between PostgreSQL TIMESTAMPTZ and TIMESTAMP?

TIMESTAMPTZ (TIMESTAMP WITH TIME ZONE): PostgreSQL normalises the value to UTC on storage. On retrieval, it converts to the session timezone (set it to UTC for consistency). Always stores the actual moment in time — no ambiguity. TIMESTAMP (WITHOUT TIME ZONE): stores the literal date and time as entered, with no timezone information. PostgreSQL doesn't store or convert any offset. This is suitable for truly timezone-neutral data (a historical record that just says '1815-06-18 11:00') but dangerous for application timestamps (who created this record? in what timezone?). Rule: use TIMESTAMPTZ for everything except LocalDate/LocalTime columns.

Q6. How does MySQL TIMESTAMP differ from PostgreSQL TIMESTAMPTZ?

MySQL TIMESTAMP stores in UTC and converts on insert/select using the server's or session's timezone. Two problems: (1) Year 2038 problem — MySQL TIMESTAMP is stored as a 32-bit Unix timestamp, overflowing on 2038-01-19. MySQL 8.0 partially addresses this. (2) Session timezone must be set explicitly (SET time_zone = '+00:00') or results are server-locale dependent. MySQL DATETIME stores no timezone at all — literal value. For MySQL: use DATETIME + application-layer UTC convention, or use MySQL 8 with TIMESTAMP for < 2038 dates. PostgreSQL's TIMESTAMPTZ is superior.

Q7. How should you set up Hibernate/JPA for correct UTC timezone handling?

Set spring.jpa.properties.hibernate.jdbc.time_zone=UTC in application.properties. This tells Hibernate to use UTC when reading and writing timestamps via JDBC, overriding the JVM default timezone. Without this: if the server's JVM timezone is 'Europe/Berlin' (+01:00), Hibernate sends timestamps shifted by 1 hour, causing silent data corruption. Also: use java.time types in entities — Instant maps to TIMESTAMPTZ, LocalDate maps to DATE. Avoid java.util.Date and Calendar in entity fields — they're mutable and timezone-problematic.

Q8. How do you extract a user's timezone and store it safely?

Collection: (1) Profile setting — let the user select from a list of IANA timezone names. (2) Auto-detect from browser: Intl.DateTimeFormat().resolvedOptions().timeZone returns the IANA ID. Store as VARCHAR(50) with the IANA ID string. Retrieval: ZoneId.of(user.getTimezoneId()) — wrap in try-catch for DateTimeException in case the stored value becomes invalid (IANA names occasionally change: 'Asia/Calcutta' was renamed to 'Asia/Kolkata'). Use ZoneId.of(id, ZoneId.SHORT_IDS) for legacy 3-letter codes. Never store raw offsets as '+05:30' alone — offsets change with DST and the ZoneId preserves that

history.

Q9. What is the year 2038 problem and which Java/database types are affected?

Unix time (time_t) is stored as a signed 32-bit integer counting seconds since 1970-01-01. It overflows on 2038-01-19T03:14:07Z. Affected: MySQL TIMESTAMP column (uses 32-bit Unix timestamp internally). Not affected: PostgreSQL TIMESTAMPTZ (uses 64-bit), Java Instant (64-bit long nanoseconds), java.time types. For applications with data or events beyond 2038 (insurance policies, mortgages, infrastructure expiry), audit MySQL TIMESTAMP columns and migrate to BIGINT (epoch ms) or DATETIME with application-layer UTC convention.

Q10. How do you make timezone-sensitive code testable?

Inject java.time.Clock as a Spring @Bean. In production: Clock.systemUTC(). In tests: Clock.fixed(Instant.parse('2024-01-15T10:00:00Z'), ZoneOffset.UTC) — all Instant.now(clock) calls in the service return this fixed value. This makes tests reproducible and timezone-independent. Test DST transitions: fix the clock to just before a DST transition (e.g., 2024-03-10T06:59:59Z = 1:59am EST) and assert the service handles the gap correctly. Without Clock injection, you'd need to mock static Instant.now() with Mockito's mockStatic, which is fragile.

Q11. How should a REST API handle timezone in request and response?

Request: accept ISO 8601 with offset (2024-01-15T10:30:00-05:00). Parse to OffsetDateTime. If a local time is submitted without offset, require a separate timezone header or query parameter (X-User-Timezone: America/New_York) and combine. Response: return UTC ISO 8601 (2024-01-15T15:30:00Z) — clients convert to their local timezone. Optionally also include the local time for UX convenience. Never return locale-formatted dates like 'Mon Jan 15 10:30 EST 2024' — ambiguous, hard to parse, locale-dependent. Document the timezone convention in the API spec (OpenAPI: format: date-time = ISO 8601).

Q12. What is TimeZone.getDefault() and why is it risky in server-side code?

TimeZone.getDefault() (or ZoneId.systemDefault()) returns the JVM's default timezone, set at startup from the OS timezone or -Duser.timezone JVM argument. Risks: (1) Servers are often in UTC, developers' laptops in local timezones — code works locally but silently produces wrong results on the server. (2) Two microservices on servers in different regions have different defaults. (3) The default can be changed at runtime (TimeZone.setDefault()) — a library could corrupt your code. Fix: always pass an explicit ZoneId or ZoneOffset. Use ZoneOffset.UTC or ZoneId.of('America/New_York') — never the system default in business logic.

Q13. Describe how Joda-Time relates to java.time and how to migrate.

Joda-Time (2002–2014) was the dominant datetime library before Java 8. Stephen Colebourne, its creator, led the Java 8 java.time (JSR-310) design, making the APIs very similar. Joda-Time is now in maintenance mode — the author recommends migrating to java.time. Migration: org.joda.time.DateTime → ZonedDateTime, Joda LocalDate → java.time LocalDate (same concept), Joda Instant → java.time Instant. Bridge method: jodaDateTime.toDate().toInstant(). Key difference: java.time ZoneId uses the JDK-bundled IANA database updated with JDK updates, while Joda-Time has its own database updated separately.

Q14. How does JDBC 4.2 improve timezone handling compared to older JDBC versions?

JDBC 4.2 (Java 8) added native support for java.time types: PreparedStatement.setObject(n, localDate) accepts LocalDate, LocalTime, LocalDateTime, OffsetDateTime, OffsetTime directly. ResultSet.getObject(n, OffsetDateTime.class) returns OffsetDateTime. Before 4.2: only java.sql.Date, java.sql.Time, java.sql.Timestamp were supported — Timestamp includes a timezone that interacts with the JVM default timezone in surprising ways. PreparedStatement.setTimestamp(n, ts, cal) with a Calendar in UTC was required to force UTC behaviour. JDBC 4.2 eliminates this complexity.

Q15. What is the 'fall back' ambiguity problem in audit logging and how do you prevent it?

During DST fall-back, local clock time 1:30am appears twice. An audit log entry showing '2024-11-03 01:30:00' is ambiguous — was it during EDT or EST? An attacker could manipulate records at this time knowing the ambiguity. Prevention: always log in UTC (Instant.now().toString() → '2024-11-03T05:30:00Z'). The UTC timestamp is unambiguous. For display: convert to the user's timezone with the offset ('2024-11-03T01:30:00-04:00' or '-05:00'). Never log in local time without an explicit offset. This also applies to financial transaction records.

Q16. How do recurring scheduled jobs handle DST transitions?

For jobs like 'run every day at 2am New York': (1) Store the schedule as 'CRON 0 2 * * * America/New_York' (zone-aware cron). Spring's @Scheduled with zone attribute or Quartz with TimeZone handles this. (2) On spring-forward, 2am doesn't exist — Quartz fires at 3am (configurable). (3) On fall-back, 2am happens twice — the job fires once (Quartz deduplicates). For absolute-time jobs ('run at epoch 1704067200'): use Instant — no DST issue. For once-daily jobs like 'report at midnight': store as LocalTime + ZoneId and recompute next firing time after each DST transition.

Q17. How do you implement timezone-aware date range queries in Spring Data JPA?

A user queries 'show me all orders from today' — 'today' is relative to their timezone. Strategy: (1) Accept the date as LocalDate from the client (the user's intended date, e.g., 2024-01-15). (2) Convert to UTC range: startInstant = LocalDate.atStartOfDay(userZoneId).toInstant(); endInstant = LocalDate.plusDays(1).atStartOfDay(userZoneId).toInstant(). (3) Query: @Query('SELECT o FROM Order o WHERE o.createdAt >= ?1 AND o.createdAt < ?2'). This correctly captures all orders between midnight and midnight in the user's timezone, regardless of what UTC range that maps to.

Q18. What are the risks of using java.util.Date and Calendar in modern Java code?

java.util.Date: (1) Mutable — can be changed after construction, causing thread-safety issues. (2) Methods like getYear() return years since 1900 (getYear() for 2024 = 124). (3) toString() uses JVM default timezone, making logs environment-dependent. Calendar: (1) Mutable and verbose. (2) Month is 0-indexed (January = 0). (3) Operations are not fluent. Both: no clear distinction between instants and local dates. Fix: use java.time everywhere. For legacy API compatibility: convert at the boundary only with Date.from(instant) and date.toInstant(). Suppress PMD/SpotBugs warnings that flag java.util.Date usage.

Q19. Describe a production bug caused by timezone handling and its fix.

Classic bug: an e-commerce site stores order timestamps in the server's local time (America/Los_Angeles, -08:00). A night batch job runs at midnight server time processing 'today's orders': SELECT * WHERE created_date = CURDATE(). For a customer in New York (+3h ahead), their order placed at 11pm ET = 8pm PT appears in the 'today' batch — correct. But after the company moves servers to UTC, 11pm ET = 2024-01-16T04:00:00Z — the batch for Jan 15 misses it. Fix: (1) Migrate all timestamps to TIMESTAMPTZ/UTC. (2) Use the user's timezone to define 'today'. (3) Test batch jobs with orders near midnight in multiple timezones.

Q20. How does the IANA timezone database relate to Java ZoneId and JDK updates?

The IANA Time Zone Database (tzdata) tracks every timezone rule change worldwide — governments regularly change DST rules, abolish DST, or change offsets. Examples: Morocco changed DST in 2018; Russia abolished DST in 2014; Samoa shifted the international date line in 2011. Java's ZoneId rules are bundled in the JDK. When IANA publishes updates, Oracle bundles them in JDK patch releases. For time-sensitive applications: keep the JDK patched. Alternatively, use the TZUpdater tool to update timezone data without upgrading the full JDK. Spring Boot's tzdata-jakarta dependency can also provide updated timezone data.